

❖ Algorithms and Data Structures

Exercise 2: E-commerce Platform Search Function

- Code:

```
using System;

class Product {
    public int ProductId { get; set; }
    public string ProductName { get; set; }
    public string Category { get; set; }

    public Product(int productId, string productName, string category) {
        ProductId = productId;
        ProductName = productName;
        Category = category;
    }

    public override string ToString() {
        return $"ID: {ProductId}, Name: {ProductName}, Category: {Category}";
    }
}

class SearchFunction {
    public static int LinearSearch(Product[] products, string name) {
        for (int i = 0; i < products.Length; i++) {
            if (products[i].ProductName.Equals(name, StringComparison.OrdinalIgnoreCase)) {
                return i;
            }
        }
        return -1;
    }

    public static int BinarySearch(Product[] products, string name) {
        int left = 0, right = products.Length - 1;
        while (left <= right) {
            int mid = left + (right - left) / 2;
            int cmp = string.Compare(products[mid].ProductName, name, StringComparison.OrdinalIgnoreCase);
            if (cmp == 0)
                return mid;
            else if (cmp < 0)
                left = mid + 1;
            else
                right = mid - 1;
        }
        return -1;
    }
}

class Program {
    static void Main(string[] args) {
        Product[] products = {
            new Product(1, "Apple", "Fruits"),
            new Product(2, "Banana", "Fruits"),
            new Product(3, "Carrot", "Vegetables"),
            new Product(4, "Mango", "Fruits")
        };

        Console.WriteLine("=== Linear Search ===");
        int index = SearchFunction.LinearSearch(products, "Carrot");
        Console.WriteLine(index != -1 ? $"Found: {products[index]}" : "Product not found");

        Array.Sort(products, (p1, p2) => p1.ProductName.CompareTo(p2.ProductName));

        Console.WriteLine("\n=== Binary Search ===");
        index = SearchFunction.BinarySearch(products, "Mango");
        Console.WriteLine(index != -1 ? $"Found: {products[index]}" : "Product not found");
    }
}
```

- Output:

```
=== Linear Search ===
Found: ID: 3, Name: Carrot, Category: Vegetables

=== Binary Search ===
Found: ID: 4, Name: Mango, Category: Fruits
```

- **Analysis:**

- Linear Search: **$O(n)$**
- Binary Search: **$O(\log n)$** (on sorted data)
- Use Binary Search for performance-critical scenarios with sorted data.

Exercise 7: Financial Forecasting

- Code:

```
using System;

class Program {
    static double PredictFutureValue(double currentValue, double growthRate, int years) {
        if (years == 0) return currentValue;
        return PredictFutureValue(currentValue * (1 + growthRate), growthRate, years - 1);
    }

    static double PredictUsingLoop(double currentValue, double growthRate, int years) {
        for (int i = 0; i < years; i++) {
            currentValue *= (1 + growthRate);
        }
        return currentValue;
    }

    static void Main(string[] args) {
        double initial = 10000;
        double rate = 0.1;
        int years = 5;

        double futureRecursive = PredictFutureValue(initial, rate, years);
        double futureIterative = PredictUsingLoop(initial, rate, years);

        Console.WriteLine($"Future Value (Recursive): {futureRecursive}");
        Console.WriteLine($"Future Value (Iterative): {futureIterative}");
    }
}
```

- Output:

```
Future Value (Recursive): 16105.100000000002
Future Value (Iterative): 16105.100000000002
```

- Analysis:

- Recursive: **O(n)** time, **O(n)** space (stack)
- Optimized (iterative): **O(n)** time, **O(1)** space
- Use loops or memoization to reduce stack overhead.